

MINOR PROJECT
ON
ASHTAVINAYAK CITY SECURITY MANAGEMENT
SYSTEM

Submitted in partial fulfillment of the requirements for
the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE WITH
DATA SCIENCE

Submitted by
MAYUR MOHITE

Y24ADS472



Department of Data Science
BAPATLA ENGINEERING COLLEGE
(Autonomous)
(Affiliated to Acharya Nagarjuna University)
BAPATLA-522101
Andhra Pradesh
INDIA



BAPATLA ENGINEERING COLLEGE, BAPATLA
(AUTONOMOUS)
Alternative Assessment Test – II (MINI PROJECT)

A.Y: 2026-27

Regd. No.

Marks:

Class: **III/IV B. Tech I-SEM**
Subject: **Software Engineering (24DS504)**

Max. Marks: **10 M**

1. Develop any one Web based Software application and describe the following things?

Sol: - SDLC life cycle diagram with description

 Requirement Analysis (SRS)

 Design Analysis (Min. four UML diagrams- Class, Use Case, Sequential, Activity)

 Coding Analysis (Sample code with web pages)

 Testing Analysis (White Box and Black Box testing)

CONTENTS

S.NO	CONTENTS	PAGE NO
1	Software Development Life Cycle	4–6
2	Introduction	7–13
3	Requirement Analysis (SRS)	14–19
4	Design Analysis	20–26
5	Coding Analysis	27–31
6	Testing Analysis	32–36
7	Output Screens / Results	37–42

1. SOFTWARE DEVELOPMENT LIFE CYCLE:

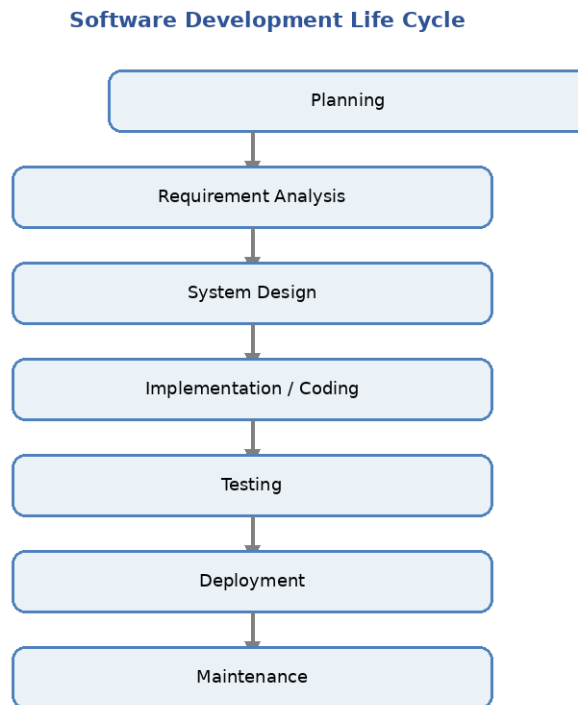


Figure: SDLC Life Cycle for Ashtavinayak City Security Management System

1.1 Planning:

Planning is the first phase of the Software Development Life Cycle for the Ashtavinayak City Security Management System. In this phase, the project problem, objectives, users, scope, resources, risks, technology choices and feasibility are identified. The main problem considered is the difficulty of managing visitor entry, resident approval, exit tracking and historical records through a purely manual process.

Main planning activities include identifying stakeholders, defining the visitor lifecycle, selecting Python/Flask, SQLite, HTML/CSS and Telegram Bot API, estimating resources, and identifying risks such as unauthorized access, database failure and loss of Telegram connectivity.

1.2. Requirement Analysis:

In Requirement Analysis, the needs of the people who use or interact with the system are collected and converted into measurable software requirements. Security Guards need controlled login, visitor registration, validation, approval tracking, exit processing and history. Residents need a secure way to receive and decide visitor approval requests through Telegram. The Boss/Administrator

needs overall records and statistics.

The requirements are classified into functional and non-functional requirements and documented in the SRS. The SRS becomes the reference for design, implementation and testing and provides traceability between user needs, system functions and test cases.

1.3. System Design:

During System Design, the requirements from the SRS are converted into a technical blueprint. The project design defines the Flask application structure, protected routes, session-based authorization, SQLite database tables, Telegram integration, user interfaces, visitor-status lifecycle and the interactions among Security, Resident and Administrator roles.

The design also identifies the principal data entities, modules and interfaces. UML models are prepared to represent actors, classes, interactions and the complete activity flow of the visitor-management process.

1.4. Implementation / Coding:

The Implementation / Coding phase converts the approved design into working software. The project is implemented using Python and Flask for application logic, HTML/CSS for web pages, SQLite for persistent storage and Telegram Bot API for resident approval communication. The system is divided into modules so that authentication, registration, approval, exit and history functions can be implemented and tested systematically.

1.5. Testing:

Testing verifies that the implemented system satisfies the specified requirements. The project uses White Box Testing to examine important program logic and branches, and Black Box Testing to verify visible behaviour through inputs and outputs. Authentication, validation, visitor registration, Telegram authorization, approval/rejection, exit processing, history and integration are included in testing.

1.6. Deployment:

Deployment makes the developed application available in its intended demonstration environment. For this project, deployment is local: the user opens the project folder in a terminal and runs `python app.py`. The Flask server starts on port 5000 and the configured browser-opening function opens Microsoft Edge when available, otherwise Google Chrome, otherwise the default browser.

1.7. Maintenance:

Maintenance is the continuing SDLC phase after deployment. For this project, maintenance includes correcting defects, updating validation rules, maintaining resident and visitor data, updating Telegram integration, improving interfaces and adding future capabilities such as QR-based visitor passes, analytics, cloud deployment and additional notification channels.

ASHTAVINAYAK CITY SECURITY MANAGEMENT SYSTEM

2. INTRODUCTION

2.1 Background

Residential societies receive visitors throughout the day, including guests, service personnel, delivery staff and other temporary visitors. Security personnel must record the visitor's identity, destination and purpose, confirm the resident being visited and determine whether the visitor is permitted to enter. When these activities are handled manually, maintaining a consistent record of every visitor and retrieving earlier information can become difficult.

The Ashtavinayak City Security Management System is developed as a web-based solution for this visitor-management process. The system provides a structured workflow beginning with Security login and visitor registration and continuing through validation, resident identification, approval, entry, exit and history. The project therefore brings the main visitor-management activities into one application.

2.2 Project Overview

The project is implemented as a web application using Python and Flask. HTML and CSS are used for the user interface, SQLite is used for persistent storage, and the Telegram Bot API is used to communicate visitor approval requests to the connected resident. The system contains separate operational paths for Security users and the Boss/Administrator.

The application maintains the visitor lifecycle through the statuses Pending, Inside, Rejected and Exited. A visitor first becomes Pending after valid registration. A resident decision through Telegram either moves the visitor to Inside or Rejected. An approved visitor can later be marked Exited by authorized Security personnel. These status changes are retained in the database with relevant timestamps.

2.3 Purpose of the Project

The purpose of the project is to provide a systematic and traceable method for managing residential visitor entry and exit. It supports authorized access, visitor-data validation, resident approval, entry and exit recording, dashboard information and historical records. The system is intended as a software-based

visitor-management solution and does not directly control physical gates or biometric devices.

2.4 Problem Statement

A manual visitor process can involve handwritten registers, repeated data entry and informal communication with residents. Such a process may make it difficult for Security personnel to know whether a particular visitor is still waiting, has been approved, is currently inside, has been rejected or has already exited. It can also make historical visitor information harder to retrieve.

The project addresses this problem by creating a centralized workflow. Security personnel enter visitor details through a web form. The application validates the mobile number, associates the visitor with the selected flat and resident, creates a Pending record and sends the approval request through Telegram. The resident's decision is then reflected in the visitor status.

2.5 Need for the System

The system is needed to provide a consistent procedure at the society entrance. The application separates the responsibilities of Security, Resident and Boss/Administrator. Security handles operational entry and exit tasks, the Resident provides the approval decision, and the Boss/Administrator can view overall visitor records and statistics.

The need also arises from the requirement to preserve visitor lifecycle information. The database stores visitor identity, purpose, destination flat, resident information, status, approval or rejection time, entry time, exit time and the Security users associated with entry and exit operations. This information supports later history and traceability.

2.6 Existing Manual Process

In a manual process, the Security Guard may write visitor information in a register and contact the resident separately. The resident's response may not be directly connected to the visitor record, and later searching for a visitor's entry or exit information can require checking earlier records manually. These limitations motivate the centralized application developed in this project.

2.7 Proposed System

The proposed system connects registration, validation, resident lookup, approval communication and status tracking. It creates a Pending record before approval,

accepts a Telegram decision only from the account connected to the relevant flat, and updates the visitor record according to the decision. The application also provides Exit and History pages so that the visitor lifecycle remains available after the initial registration.

2.8 Objectives

The main objectives of the Ashtavinayak City Security Management System are:

1. To provide controlled login for authorized Security and Boss/Administrator users.
2. To register visitor information such as name, mobile number, purpose, destination flat, resident and expected duration.
3. To validate the visitor mobile number according to the selected country-code rule.
4. To identify the resident and flat associated with the visitor request.
5. To create a Pending visitor request before resident approval.
6. To send visitor approval information to the connected resident through Telegram.
7. To accept APPROVE or DISAPPROVE actions only from the Telegram account associated with the relevant flat.
8. To record approval, rejection, entry and exit timestamps.
9. To allow Security users to search current Inside visitors and mark their exit.
10. To provide visitor history and dashboard information for later reference.
11. To provide resident-management and administrative functions required by the application.
12. To automatically open the local web application in a supported browser when the Flask server is started.

2.9 Project Goals

The project goal is to demonstrate a complete Software Engineering workflow in which requirements are translated into design, implementation and testing. The application combines authentication, database operations, web interfaces and external Telegram communication in one visitor-management process.

2.10 Scope of the Project

The project covers the software activities required to manage visitors in a residential society. It includes Security login, Boss/Administrator login, visitor registration, mobile-number validation, resident and flat selection, Pending request creation, Telegram approval or disapproval, entry recording, exit processing, visitor history and dashboard information.

The scope also includes resident management, database storage and automatic browser opening for the local Flask application. The project is designed around a controlled local demonstration environment and the visitor-management workflow represented in the SRS.

The project does not include payment management, accounting, maintenance billing, biometric hardware, physical gate control or a complete hardware-based access-control system. Telegram approval depends on internet connectivity and a correctly connected resident Telegram account.

2.11 Users of the System

Security Guard: The Security user logs in to the application, registers visitors, validates information, selects the resident and flat, checks approval status, performs entry-related operations, searches current visitors and records exits.

Resident: The resident uses the connected Telegram account to receive visitor information and select APPROVE or DISAPPROVE for the visitor request associated with the relevant flat.

Boss / Administrator: The authorized administrator can view overall visitor records and statistics through the administrative dashboard.

2.12 Major Features

The major features include Security Login, Administrator Login, Security Dashboard, Visitor Registration, Phone Validation, Resident Lookup, Pending Approval, Telegram Approval, Entry Recording, Exit Search, Mark Exit, Visitor History, Resident Management, Boss Dashboard and Automatic Browser Opening.

2.13 Visitor Lifecycle

The visitor lifecycle begins when Security logs in and registers a visitor. After validation and resident selection, the application creates a Pending request. The

approval request is sent through Telegram. If the resident approves, the visitor changes from Pending to Inside and approval/entry time is recorded. If the resident disapproves, the visitor changes from Pending to Rejected and rejection time is recorded.

For an approved visitor, the Exit page displays the visitor while the status is Inside. When Security performs Mark Exit, the status changes to Exited and the exit time and responsible Security user are stored. The completed record remains available through Visitor History.

2.14 Roles and Responsibilities

Security is responsible for registration and operational visitor processing. The Resident is responsible for the approval decision. The Boss/Administrator is responsible for viewing overall information and statistics. The web application coordinates these activities and the SQLite database preserves the corresponding records.

2.15 Security and Access Control

Protected routes require authorized user sessions. Security and Boss/Administrator access are separated according to the role stored in the application session. Telegram approval is also checked against the resident associated with the visitor's destination flat. Application secrets, Telegram bot tokens and personal identifiers should be protected from unnecessary exposure.

2.16 Data Traceability

The system maintains visitor information and important lifecycle timestamps so that an individual record can be followed from creation to final exit or rejection. This traceability supports the History page and provides a basis for testing the visitor-status transitions defined in the requirements.

2.17 Technology Overview

Python and Flask provide the backend application logic and routing. HTML and CSS provide the browser-based user interface. SQLite provides persistent storage for resident and visitor information. Telegram Bot API provides the external communication path for resident approval. The local application is executed using python app.py and runs through the Flask development server.

The demonstrated browser-opening function checks available Microsoft Edge paths first, then Google Chrome paths, and finally uses the default system

browser. This supports a simple local demonstration workflow after the Flask server is started.

2.18 System Environment

The project requires a computer or laptop capable of running Python and a modern web browser. The demonstrated setup uses a Windows environment, Flask and the required Python packages, SQLite support and internet connectivity for Telegram communication. A connected Telegram account is required for the resident approval workflow.

2.19 Database Overview

The application uses two principal data entities: Resident and Visitor. Resident records contain resident identity, flat, phone and Telegram connection information. Visitor records contain visitor identity, mobile number, purpose, destination flat, resident information, duration, status and the timestamps and Security information associated with approval, entry and exit.

2.20 Interfaces

The main interfaces are Login, Security Dashboard, Register Visitor, Approval, Telegram approval message, Exit, History and Boss Dashboard. Together these interfaces represent the operational workflow and provide access to the functions defined in the SRS.

2.21 Benefits of the Proposed System

The proposed application provides a centralized visitor record instead of separating registration, approval and exit information across unrelated manual records. It makes the resident decision explicit, preserves visitor status and timestamps, and provides a History page for later reference. The dashboard also provides an operational view of visitor counts and current records.

2.22 Limitations

The current project is a software-based local Flask application. Telegram approval depends on internet connectivity and correct Telegram configuration. Visitor records depend on the availability of the SQLite database. The system does not directly operate a physical gate or biometric device, and the demonstrated deployment is local rather than a cloud-hosted production environment.

2.23 Future Direction

The existing project materials identify future capabilities such as QR-based visitor passes, analytics, cloud deployment and additional notification channels. These capabilities can be considered extensions of the current visitor-management workflow rather than part of the present implementation.

2.24 Organization of the Report

After this seven-page introduction, the report continues with the Software Development Life Cycle and then Requirement Analysis in the form of the Software Requirements Specification (SRS). The SRS defines the purpose, scope, terminology, feasibility, requirements, constraints, interfaces, hardware/software needs and data-storage requirements. The report then continues with Design Analysis, UML diagrams, Coding Analysis, Output Screens / Results, Testing Analysis, Conclusion, Future Scope, References and the project repository.

2.25 Introduction Summary

The Ashtavinayak City Security Management System provides a structured visitor-management workflow for a residential society. Its core process connects Security registration with resident approval through Telegram and maintains the resulting entry, exit and history information. The following sections convert this overview into formal SDLC and SRS documentation and then describe the technical design, implementation and testing of the system.

3. REQUIREMENT ANALYSIS (SRS)

3.1 Purpose

The purpose of the Ashtavinayak City Security Management System is to provide a structured web-based system for managing visitor entry and exit in a residential society. The application allows authorized security personnel to register visitors, validate details, send resident approval requests through Telegram, record entry and exit events, and maintain visitor history.

3.2 Scope

The system covers Security Guard login, visitor registration, mobile-number validation, resident and flat identification, Pending request creation, Telegram-based resident approval, automatic entry recording after approval, rejection recording, exit search, exit recording, visitor history and Security/Boss dashboards. It does not claim to provide payment management, accounting, maintenance billing, biometric hardware or full physical access-control integration.

3.3 Definitions, Acronyms, and Abbreviations

- UI: User Interface.
- SRS: Software Requirements Specification.
- UML: Unified Modeling Language.
- API: Application Programming Interface.
- Flask: Python web framework used to implement the web application.
- SQLite: Lightweight relational database used for resident and visitor records.
- Telegram Bot API: Messaging interface used for resident approval requests and callback actions.
- Security Guard: Authorized user who registers visitors and manages entry/exit operations.
- Resident: Person associated with a flat who approves or disapproves a visitor request.
- Boss / Administrator: Authorized user who can view overall visitor records and statistics.

3.4 Planning Phase

The planning phase identifies the target users, project objectives, major modules, required technologies, data requirements and external communication dependency. The principal modules are authentication, dashboard, visitor registration, validation, resident approval, entry/exit tracking and history.

3.5 Feasibility Study

3.5.1 Technical Feasibility

The project is technically feasible because it uses Python, Flask, SQLite, HTML/CSS and Telegram Bot API, all of which are suitable for a web-based visitor-management prototype. The application can run locally on a standard computer with Python and a modern browser.

3.5.2 Operational Feasibility

The workflow matches the actual gate process: Security logs in, registers a visitor, selects the resident, waits for the resident decision, records entry for an approved visitor, and later records exit. This makes the system suitable for day-to-day demonstration and controlled society use.

3.5.3 Economic Feasibility

The core application uses Python, Flask and SQLite, which do not require a paid database server for the project prototype. The major external dependency is Telegram connectivity for approval communication.

3.5.4 Schedule Feasibility

The system is divided into manageable SDLC activities: planning, requirements, design, coding, testing and demonstration. The modular structure supports incremental development and testing.

3.6 Requirement Analysis

Requirement analysis converts the visitor-management workflow into specific system requirements. The analysis identifies users, inputs, processing rules, database information, approval conditions, status transitions, interfaces and security controls.

- ✓ Analyze Security Guard, Resident and Boss/Administrator workflows.
- ✓ Define visitor input fields and validation rules.
- ✓ Define resident and flat selection rules.
- ✓ Define Pending, Inside, Rejected and Exited status transitions.
- ✓ Define Telegram approval and authorization rules.
- ✓ Define entry, exit, approval and rejection timestamps.
- ✓ Define search, history and dashboard information.
- ✓ Define database storage for residents and visitors.
- ✓ Define browser and Telegram interfaces.
- ✓ Define access-control and configuration requirements.

3.7 Product Functions

- ✚ Security Login – authenticate authorized security users.
- ✚ Security Dashboard – display visitor counts and operational information.
- ✚ Visitor Registration – capture name, mobile, purpose, flat, resident and duration.
- ✚ Phone Validation – validate country code and required digit length.
- ✚ Resident Lookup – associate the visitor with the selected resident and flat.
- ✚ Pending Request – create a visitor record with Pending status.
- ✚ Telegram Approval – send visitor details to the connected resident.
- ✚ Resident Decision – accept APPROVE or DISAPPROVE through the authorized Telegram account.
- ✚ Entry Recording – change approved visitors to Inside and store approval/entry time.
- ✚ Exit Processing – search current Inside visitors and mark them Exited.
- ✚ History – display stored visitor records and status history.
- ✚ Boss Dashboard – provide overall records and statistics to the authorized administrator.
- ✚ Automatic Browser Opening – open the local application in Edge or Chrome when python app.py is executed.

3.8 User Characteristics

- Security Guard: Basic computer and browser knowledge; performs login, registration, visitor search, entry-related and exit operations.
- Resident: Uses the connected Telegram account to approve or disapprove visitor requests for the relevant flat.
- Boss / Administrator: Uses the administrative dashboard to view overall visitor records and statistics.

3.9 Constraints

- ✓ Telegram approval depends on internet connectivity and a correctly connected resident Telegram account.
- ✓ Persistent visitor records depend on SQLite database availability.
- ✓ Protected operations require authorized user sessions.
- ✓ Personal visitor and resident information must be protected from unauthorized access.
- ✓ The current project is software-based and does not directly control physical gates or biometric devices.
- ✓ Local demonstration depends on the Flask application being running on the host computer.

3.10 Specific Requirements

3.10.1 Functional Requirements

ID	Requirement	Description
FR-01	Security Login	Allow authorized security users to log in and access protected pages.
FR-02	Administrator Login	Allow the authorized Boss/Administrator account to access the admin dashboard.
FR-03	Visitor Registration	Capture visitor name, mobile, purpose, flat, person to meet and duration.
FR-04	Phone Validation	Validate country code and required mobile-number length.
FR-05	Resident Lookup	Verify that the selected resident and flat exist.
FR-06	Pending Request	Create a visitor record with Pending status.
FR-07	Telegram Request	Send visitor approval details to the connected resident.
FR-08	Telegram Authorization	Accept decisions only from the Telegram account connected to the relevant flat.
FR-09	Approval	Change Pending visitor to Inside and store approval/entry time.
FR-10	Disapproval	Change Pending visitor to Rejected and store rejection time.
FR-11	Exit Search	Display visitors currently having Inside status.
FR-12	Mark Exit	Change Inside visitor to Exited and store exit time and security user.
FR-13	History	Display visitor records for later reference.
FR-14	Dashboard	Display visitor statistics and current records.
FR-15	Browser Opening	Open the local Flask application in Edge/Chrome when configured browser paths are available.

3.10.2 Non-Functional Requirements

Requirement	Description
Performance	Pages, searches and normal database operations should respond promptly for the project dataset.
Usability	The interface should provide clear forms, buttons, status labels and navigation for Security users.
Security	Protected routes require authorized sessions; Telegram decisions are checked against the relevant flat.
Reliability	Visitor status and timestamps should be stored consistently in the database.
Data Integrity	Visitor records should preserve entry, exit, approval/rejection and responsible security information.
Maintainability	Application logic, templates, database operations and configuration should remain organized.

Availability	The local web application and Telegram approval path must be available when required.
Compatibility	The web interface should work in modern desktop browsers such as Microsoft Edge and Google Chrome.
Privacy	Phone numbers, Telegram identifiers, bot tokens and application secrets should not be exposed unnecessarily.

3.11 Other Requirements

3.11.1 Interfaces

- ✚ Login Interface: Username and password fields for Security and Boss/Administrator access.
- ✚ Security Dashboard: Counts, current visitor information and navigation to operational pages.
- ✚ Register Visitor Interface: Visitor details, country code, resident/flat selection and duration.
- ✚ Approval Interface: Pending visitor requests and Telegram resend operation.
- ✚ Telegram Interface: Visitor details with APPROVE and DISAPPROVE actions.
- ✚ Exit Interface: Search/current Inside visitors and Mark Exit action.
- ✚ History Interface: Stored visitor records and lifecycle information.
- ✚ Boss Dashboard: Overall visitor records and statistics.

3.11.2 Hardware and Software Requirements

- ❖ Computer or laptop capable of running Python and a modern web browser.
- ❖ Windows environment for the demonstrated local setup.
- ❖ Python runtime with Flask and required project packages.
- ❖ SQLite database support.
- ❖ Microsoft Edge or Google Chrome for browser-based access.
- ❖ Internet connectivity for Telegram approval communication.
- ❖ Telegram application/account for the connected resident workflow.
- ❖ Development editor such as Visual Studio Code may be used for source-code editing.

3.11.3 Data Storage Requirements

The application uses SQLite database storage. The main resident table stores resident identity, flat, phone and Telegram connection information. The visitor table stores visitor identity, purpose, destination flat, resident information, duration, status, approval/rejection timestamps, entry/exit timestamps and the security users responsible for operations.

Residents Field	Type	Purpose
id	INTEGER	Primary key
name	TEXT	Resident name
flat	TEXT	Flat identifier
phone	TEXT	Resident phone
telegram_chat_id	TEXT	Connected Telegram chat identifier

Visitors Field	Type	Purpose
id	INTEGER	Primary key
name	TEXT	Visitor name
mobile	TEXT	Visitor mobile number
purpose	TEXT	Purpose of visit
flat	TEXT	Destination flat
person_to_meet	TEXT	Resident to meet
resident_phone	TEXT	Resident phone reference
duration	TEXT	Expected duration
status	TEXT	Pending / Inside / Rejected / Exited
entry_time	TEXT	Entry timestamp
exit_time	TEXT	Exit timestamp
entry_security	TEXT	Security user for entry
exit_security	TEXT	Security user for exit
approval_time	TEXT	Resident approval timestamp
rejection_time	TEXT	Resident rejection timestamp
sms_sent	INTEGER	Notification flag retained by database schema
sms_sid	TEXT	Notification reference field
created_time	TEXT	Visitor request creation time

4. DESIGN ANALYSIS

The design analysis converts the SRS into the technical structure of the Ashtavinayak City Security Management System. The design covers system architecture, module design, data design, user-interface design, overall workflow and UML representation.

4.1 System Architecture

The application follows a web-based architecture in which Security and Administrator users interact with Flask web pages. Flask handles authentication, routing, validation and database operations. SQLite stores resident and visitor records. Telegram Bot API provides the resident approval communication path.

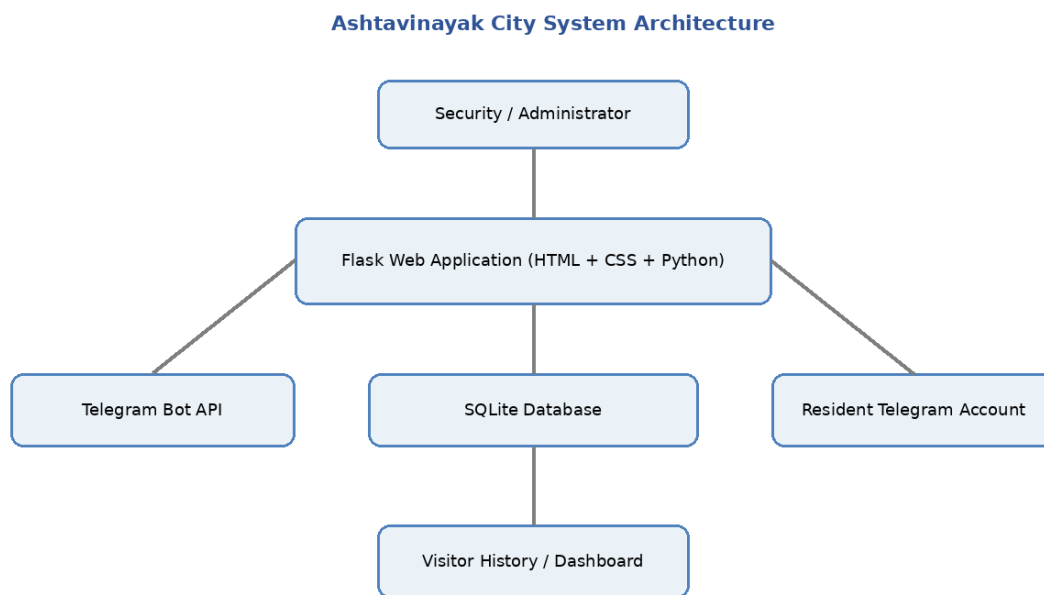


Figure 4.1: System Architecture

4.2 Model Design

Module	Functionality
Login and Authorization Module	Validates Security and Boss credentials, creates the appropriate session role and protects restricted routes.
Security Dashboard Module	Displays visitor counts and provides navigation to registration, approval, exit and history functions.
Visitor Registration Module	Collects visitor information, validates the mobile number, identifies the resident and creates a Pending request.
Resident Approval Module	Sends visitor details to Telegram and processes APPROVE/DISAPPROVE callbacks only for the authorized resident account.

Entry Module	After approval, changes visitor status to Inside and records approval and entry time.
Exit Module	Lists current Inside visitors and allows Security to mark the selected visitor as Exited.
History Module	Displays all stored visitor records for traceability and later reference.
Boss / Administrator Module	Provides overall visitor records and dashboard statistics.
Browser Opening Module	Starts the local Flask server and automatically opens Edge, Chrome or the default browser.

The Model Design divides the system into independent but connected modules. Each module has a defined responsibility and exchanges data through the Flask application and SQLite database.

1. Login and Authorization Module – validates Security and Boss credentials and creates the correct session role.
2. Security Dashboard Module – displays visitor statistics and provides navigation to the main security operations.
3. Visitor Registration Module – captures visitor details, validates input, identifies the resident and creates a Pending request.
4. Resident Approval Module – sends the request through Telegram and processes APPROVE or DISAPPROVE actions.
5. Entry and Exit Tracking Module – records approved entry and allows authorized Security to mark visitor exit.
6. History Report Module – displays stored visitor records, status, entry/exit times and responsible Security users.
7. Resident Management Module – supports adding and removing resident records.
8. Boss/Administrator Module – provides administrative visitor statistics and records.

4.3 Data Design

The database design uses two principal entities supplied by the application schema: Resident and Visitor. A resident is identified by a unique name-flat combination and may have a Telegram chat identifier. A visitor record is linked to the destination flat and resident information and progresses through the defined visitor lifecycle.

- ❖ Resident Entity → stores resident identity, flat, phone and Telegram connection.
- ❖ Visitor Entity → stores visitor details, destination, status, approval/rejection and entry/exit timestamps.

- ❖ Relationship → a resident/flat can receive multiple visitor requests over time.
- ❖ Status lifecycle → Pending → Inside → Exited for an approved visitor; Pending → Rejected for a disapproved visitor.

4.4 User Interface Design

The interface is designed around the Security workflow. The main navigation is: Login → Security Dashboard → Register Visitor → Pending Approval → Resident Telegram Decision → Inside Visitor → Exit Search → Mark Exit → Visitor History. The Boss/Administrator path is Login → Boss Dashboard → View Overall Records and Statistics.

- Login page uses username and password fields and displays an error for invalid credentials.
- Dashboard provides visitor statistics and navigation controls.
- Registration page provides visitor details and resident/flat selection.
- Approval page lists Pending requests and provides Telegram resend support.
- Exit page lists current Inside visitors and provides Mark Exit controls.
- History page presents stored visitor information in tabular form.
- Telegram messages provide APPROVE and DISAPPROVE actions to the connected resident.

4.5 Overall System Workflow



4.6 UML DESIGN

The UML design contains the four diagrams required for the Software Engineering mini project: Use Case, Class, Sequence and Activity. The diagrams are based on the actual Ashtavinayak visitor-management workflow.

4.6.1 USE CASE DIAGRAM

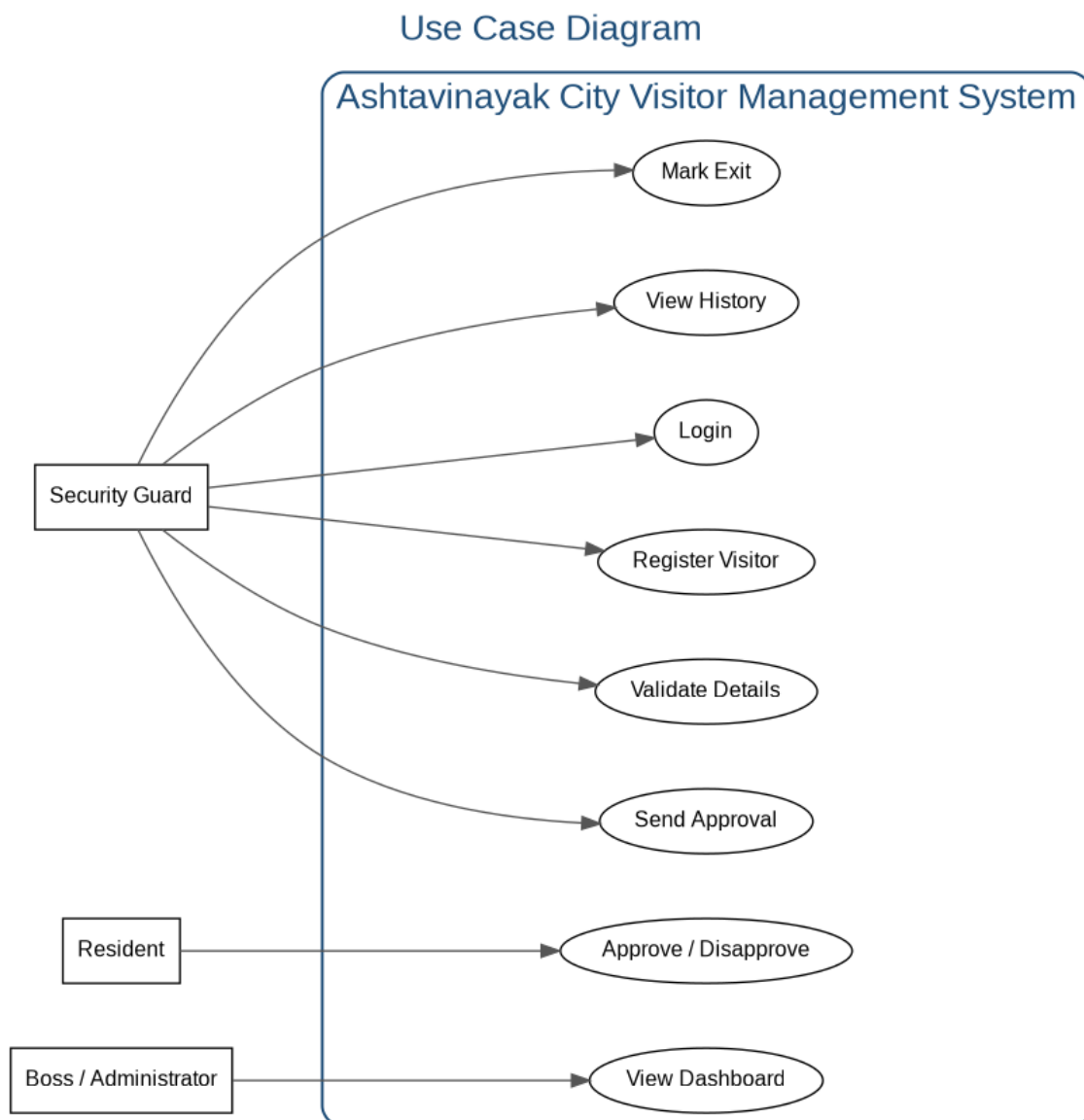


Figure 4.2: Use Case Diagram

The Use Case Diagram represents the main actors and functions. Security Guard performs login, visitor registration, validation, approval-request sending, exit and history operations. Resident performs approval/disapproval through Telegram. Boss/Administrator views the dashboard.

4.6.2 CLASS DIAGRAM

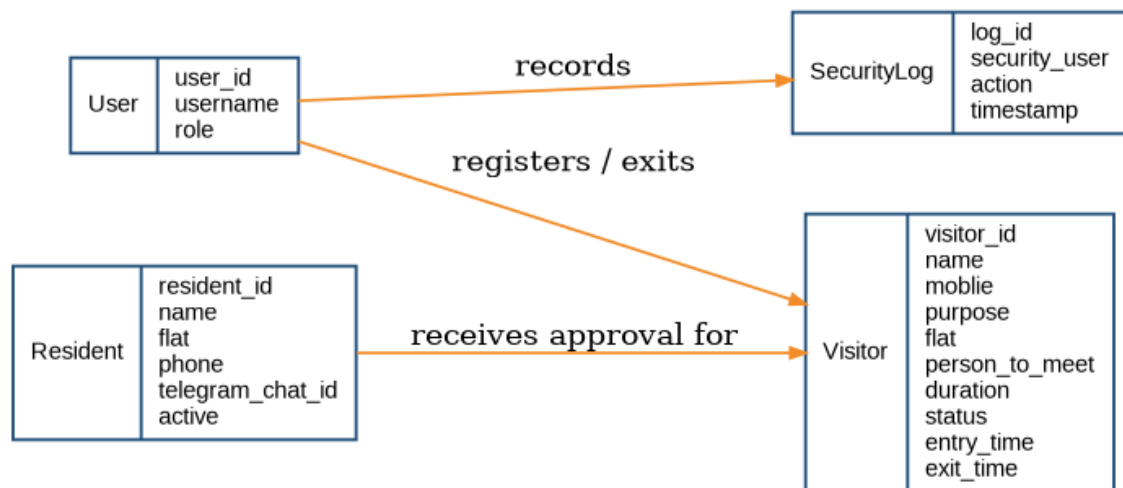


Figure 4.3: Class Diagram

The Class Diagram represents the principal data entities and their relationships. User/Security information is associated with security activity, Resident stores flat and Telegram information, and Visitor stores the visitor lifecycle and timestamps.

4.6.3 SEQUENCE DIAGRAM

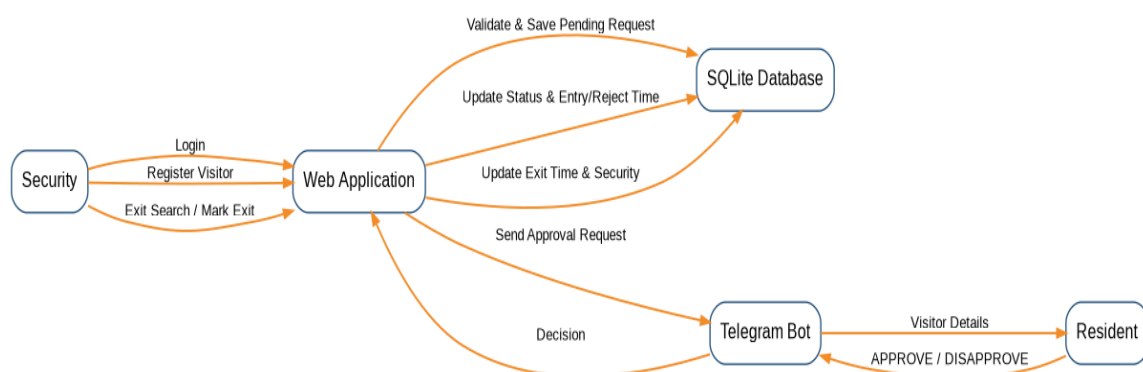


Figure 4.4: Sequence Diagram

The Sequence Diagram shows the interaction order among Security, Web Application, SQLite Database, Telegram Bot and Resident. Registration is saved as Pending, the Telegram request is sent, the resident decision is returned, the visitor status is updated, and exit information is later recorded.

4.6.4 ACTIVITY DIAGRAM

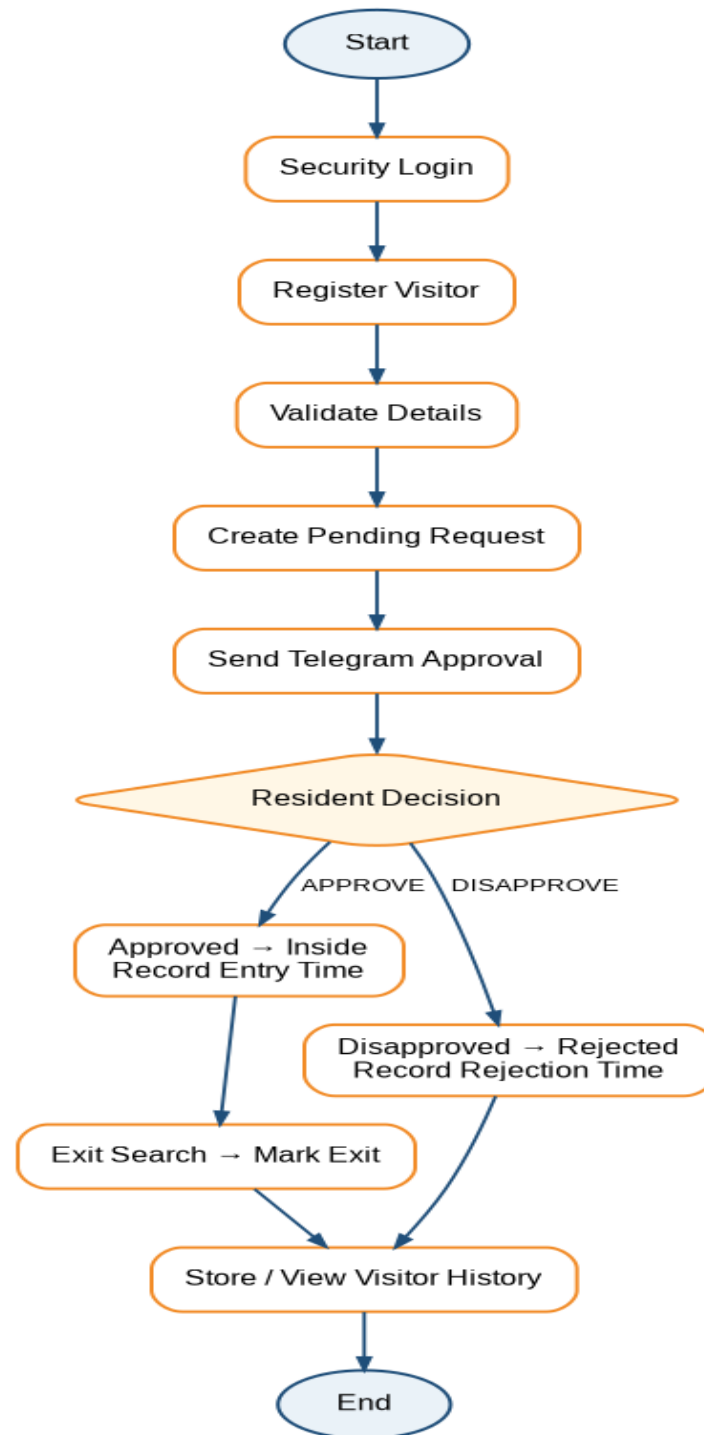


Figure 4.5: Activity Diagram

The Activity Diagram represents the end-to-end workflow from Security Login through registration, validation, Pending creation and resident decision. Approval leads to Inside and later Exit; disapproval leads to Rejected and history.

5. CODING ANALYSIS

The Coding Analysis explains how the design is implemented in the actual Flask application. The application uses Python for backend logic, Flask for web routing, SQLite for storage, HTML/CSS templates for the user interface and Telegram Bot API for resident approval communication.

5.1 Python / Flask Application Setup

The Flask application creates the web server, defines routes and maintains session-based role information. The local project is run using `python app.py`.

```
from flask import Flask, render_template, request, redirect, url_for, session
import sqlite3

app = Flask(__name__)
DATABASE = "ashtavinayak_city.db"

def get_db():
    conn = sqlite3.connect(DATABASE, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    return conn
```

This setup creates the Flask application and provides a database connection function that returns SQLite rows in a convenient dictionary-like form.

5.2 Security Login and Authorization

The login route checks the submitted username and password against the configured Security users and Boss/Administrator account. On successful authentication, the session stores the role and username. Protected routes check the role before allowing access.

```
if (
    username in SECURITY_USERS
    and password == SECURITY_USERS[username]
):
    session.clear()
    session["role"] = "security"
    session["username"] = username
    return redirect(url_for("dashboard"))
```

5.3 Visitor Registration and Validation

The registration module reads visitor name, country code, mobile number, purpose, flat, person to meet and duration. The phone validation function removes permitted formatting characters, checks that the value contains digits and verifies the exact length required for the selected country code.

```
valid, result = validate_phone_number(
    country_code,
    mobile
)

if not valid:
    return render_template(
        "register.html",
        residents=residents,
        country_codes=COUNTRY_CODES.keys(),
        error=result
    )
```

5.4 Pending Request and Database Insert

After validation and resident lookup, the application inserts a visitor record with Pending status. Entry time is not created at registration; it is created only after resident approval.

```
conn.execute("""
    INSERT INTO visitors
    (name, mobile, purpose, flat, person_to_meet,
    resident_phone, duration, status,
    entry_security, created_time)
    VALUES (?, ?, ?, ?, ?, ?, ?, 'Pending', ?, ?)
""", (
    name,
    country_code + result,
    purpose,
    resident_flat,
    resident["name"],
    resident["phone"],
    duration,
    session["username"],
    created_time,
))
```

5.5 Resident Approval Integration

The Telegram callback identifies the visitor request and checks that the Telegram chat ID is connected to the resident of the visitor's flat. APPROVE changes Pending to Inside and stores approval/entry time. DISAPPROVE changes Pending to Rejected and stores rejection time.

```
resident = conn.execute("""
    SELECT *
    FROM residents
    WHERE flat = ?
    AND telegram_chat_id = ?
    """, (visitor["flat"], chat_id)).fetchone()

if action == "approve":
    conn.execute("""
        UPDATE visitors
        SET status='Inside',
            approval_time=?,
            entry_time=?
        WHERE id=? AND status='Pending'
        """, (now, now, visitor_id))
```

5.6 Entry, Exit and History

When a visitor is approved, the visitor becomes Inside. The Exit page lists visitors with Inside status. The Mark Exit operation changes the status to Exited and stores exit time and the responsible security username. The History page displays visitor records ordered by the latest visitor ID.

```
conn.execute("""
    UPDATE visitors
    SET status='Exited',
        exit_time=?,
        exit_security=?
    WHERE id=? AND status='Inside'
    """, (now, current_security, id))
```

5.7 Automatic Web Browser Opening

For demonstration, the Flask application can automatically open the local project in a browser after python app.py is executed. The implementation checks Microsoft Edge first, then Google Chrome, and finally falls back to the

default system browser. The Flask server continues to run on <http://127.0.0.1:5000>.

```
import os
import webbrowser
from threading import Timer

def open_browser():
    url = "http://127.0.0.1:5000"

    edge_paths = [

os.path.expandvars(r"%ProgramFiles(x86)%\Microsoft\Edge\Ap
plication\msedge.exe"),

os.path.expandvars(r"%ProgramFiles%\Microsoft\Edge\Applica
tion\msedge.exe"),

os.path.expandvars(r"%LOCALAPPDATA%\Microsoft\Edge\Applica
tion\msedge.exe"),
    ]

    chrome_paths = [

os.path.expandvars(r"%ProgramFiles%\Google\Chrome\Applicat
ion\chrome.exe"),

os.path.expandvars(r"%ProgramFiles(x86)%\Google\Chrome\App
lication\chrome.exe"),

os.path.expandvars(r"%LOCALAPPDATA%\Google\Chrome\Applicat
ion\chrome.exe"),
    ]

    for browser_path in edge_paths:
        if os.path.exists(browser_path):
            webbrowser.register(
                "ashtavinayak_edge",
                None,
                webbrowser.BackgroundBrowser(browser_path)
            )
            webbrowser.get("ashtavinayak_edge").open(url)
    return
```

```

    for browser_path in chrome_paths:
        if os.path.exists(browser_path):
            webbrowser.register(
                "ashtavinayak_chrome",
                None,
                webbrowser.BackgroundBrowser(browser_path)
            )

webbrowser.get("ashtavinayak_chrome").open(url)
return

webbrowser.open(url)

Timer(1.5, open_browser).start()

app.run(
    debug=True,
    use_reloader=False,
    host="0.0.0.0",
    port=5000
)

```

5.8 Execution

Open the project folder in the terminal and run `python app.py`. The Flask server starts on port 5000. With the automatic browser-opening code enabled, the project opens in Microsoft Edge if available, otherwise Google Chrome, otherwise the system default browser.

6. TESTING ANALYSIS

Testing Analysis is performed to verify that the Ashtavinayak City Security Management System works according to the defined requirements. The major modules tested are Security Login, Administrator Login, Visitor Registration, Phone Validation, Resident Lookup, Telegram Approval, Entry, Exit, History and Dashboard.

Two primary testing techniques are used: White Box Testing and Black Box Testing. Integration Testing is also included because the project connects the web application, SQLite database and Telegram service.

Comparison Overview

Feature	Black Box Testing	White Box Testing
Focus	External behavior and user interface.	Internal code structure, logic, branches and data operations.
Tester View	Inputs are provided and visible outputs are checked.	Source-code paths, conditions and database operations are examined.
Knowledge Needed	Requirements and functional specifications.	Source code, database structure and application logic.
Project Example	Can a visitor be registered and approved successfully?	Does the approval branch update Pending to Inside and store entry time?

6.1 WHITE BOX TESTING

White Box Testing examines internal program logic, conditions, branches and execution paths. In this project it is used to verify authentication branches, validation logic, database status transitions, Telegram authorization and exit conditions.

Test Case	Code / Function Tested	Expected Result
WB01	Security login validation branch	Valid credentials should create the security session and open the dashboard.
WB02	Invalid login branch	Invalid credentials should return the login page with an error.
WB03	Phone digit validation	Non-numeric phone input should be rejected.
WB04	Phone length validation	Incorrect length for the selected country code should be rejected.
WB05	Resident lookup branch	A valid resident/flat combination should be found; invalid selection should return an error.

WB06	Pending creation branch	A valid visitor should be inserted with Pending status and created_time.
WB07	Telegram authorization branch	Only the Telegram account connected to the relevant flat should be authorized.
WB08	Approval branch	APPROVE should change Pending to Inside and store approval/entry time.
WB09	Rejection branch	DISAPPROVE should change Pending to Rejected and store rejection time.
WB10	Exit branch	Only an Inside visitor should be changed to Exited with exit time/security.

6.1.1 Structural Testing Scenarios

- Authentication Branch Coverage: Test both successful and unsuccessful Security/Boss login branches to ensure sessions are created only for valid credentials.
- Phone Validation Branch Coverage: Execute valid, empty, non-numeric and incorrect-length mobile inputs so that each validation condition is exercised.
- Visitor Status Branch Coverage: Execute APPROVE and DISAPPROVE paths and verify the corresponding Pending → Inside and Pending → Rejected transitions.
- Exit Condition Coverage: Verify that only a visitor whose status is Inside can be changed to Exited.

6.1.2 Approval Branch Example

```

if action == "approve":
    status = "Inside"
    approval_time = now
    entry_time = now
elif action in ("reject", "disapprove"):
    status = "Rejected"
    rejection_time = now
else:
    return "Unknown action"

```

White box testing ensures that each branch is executed and that the database update corresponds to the selected resident decision.

6.1.3 Database Transaction Verification

The tester verifies that an approved visitor is updated in the SQLite database with status = Inside, approval_time and entry_time, and that a visitor marked for exit is updated with status = Exited, exit_time and exit_security.

6.1.4 Exception Handling

Telegram sending and polling operations contain exception handling. Testing can intentionally simulate unavailable Telegram connectivity or a send failure and verify that the application reports the error instead of terminating the complete Flask application.

6.2 BLACK BOX TESTING

Black Box Testing verifies functionality from the user's point of view without depending on the internal source code. The tester provides inputs and checks the visible output.

Test ID	Input / Action	Expected Output	Status
BB01	Enter valid Security credentials	Security Dashboard opens	Pass
BB02	Enter invalid credentials	Error message is displayed	Pass
BB03	Register visitor with valid details	Pending visitor request is created	Pass
BB04	Enter invalid mobile number	Validation error is displayed	Pass
BB05	Select a valid resident and flat	Resident information is associated with the visitor	Pass
BB06	Open Approval page	Pending visitor request is displayed	Pass
BB07	Resident presses APPROVE in Telegram	Visitor becomes Inside and entry time is recorded	Pass
BB08	Resident presses DISAPPROVE in Telegram	Visitor becomes Rejected and rejection time is recorded	Pass
BB09	Search visitor on Exit page	Current Inside visitor is displayed	Pass
BB10	Click Mark Exit	Visitor becomes Exited and exit details are stored	Pass
BB11	Open Visitor History	Stored visitor records are displayed	Pass
BB12	Run python app.py	Flask server starts and configured browser opens automatically	Pass

6.2.1 Functional Testing Scenarios

- ✓ User Login: Verify that valid Security credentials open the dashboard and invalid credentials display an error.

- ✓ Visitor Registration: Verify that valid visitor details create a Pending request and invalid mobile data is rejected.
- ✓ Telegram Approval Flow: Verify that the resident receives visitor details and that APPROVE or DISAPPROVE produces the corresponding status.
- ✓ Entry / Exit Flow: Verify that an approved visitor is shown as Inside and later becomes Exited after Mark Exit.

6.2.2 Test Case Design Techniques Applied

- ✚ Boundary Value Analysis (BVA): Test the boundary of the mobile-number validation rule. For an Indian +91 number requiring 10 digits, test 9 digits, exactly 10 digits and 11 digits to verify rejection/acceptance at the boundary.
- ✚ Equivalence Partitioning: Divide visitor mobile input into valid and invalid classes. For example: numeric 10-digit input (valid), fewer than 10 digits (invalid), more than 10 digits (invalid), and non-numeric input (invalid).

6.3 TESTING OF MAJOR MODULES

Login Module: Valid and invalid Security/Boss credentials are tested. Valid users reach their respective dashboards, while invalid credentials are rejected.

Visitor Registration Module: Visitor name, mobile number, purpose, destination flat, resident and duration are entered. Validation and resident lookup are checked before the Pending record is created.

Telegram Approval Module: The system sends visitor details to the connected resident. The authorization branch ensures that only the Telegram account associated with the relevant flat can approve or disapprove the request.

Entry and Exit Module: Approved visitors become Inside and receive approval/entry timestamps. The Exit page lists Inside visitors, and Mark Exit records Exited status, exit time and responsible security user.

History and Dashboard Module: Stored visitor records and statistics are checked for consistency with the database status values.

6.4 INTEGRATION TESTING

Integration testing verifies the interaction among the Flask web application, SQLite database and Telegram service. The main integration path is: Security registration → database record → Telegram request → resident decision → database status update → approval status → exit recording → visitor history.

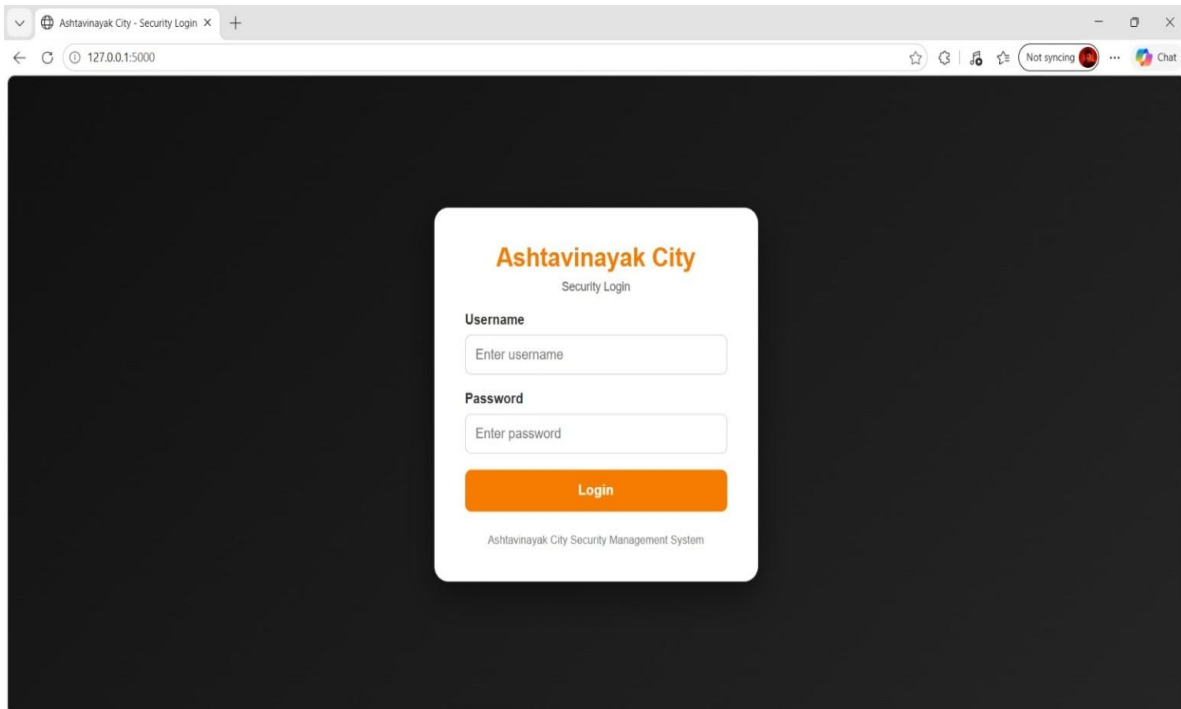
6.5 TESTING RESULT

The test cases cover the principal visitor-management workflow and its important internal branches. The project materials demonstrate successful

execution of login, dashboard, visitor registration, validation, Telegram approval, entry, exit and history operations.

7. OUTPUT SCREENS / RESULTS

The following output screens are taken from the actual Ashtavinayak City Security Management System report/project materials. They demonstrate the working user interfaces and the visitor lifecycle from authentication to history.



Ashtavinayak City
Security Login

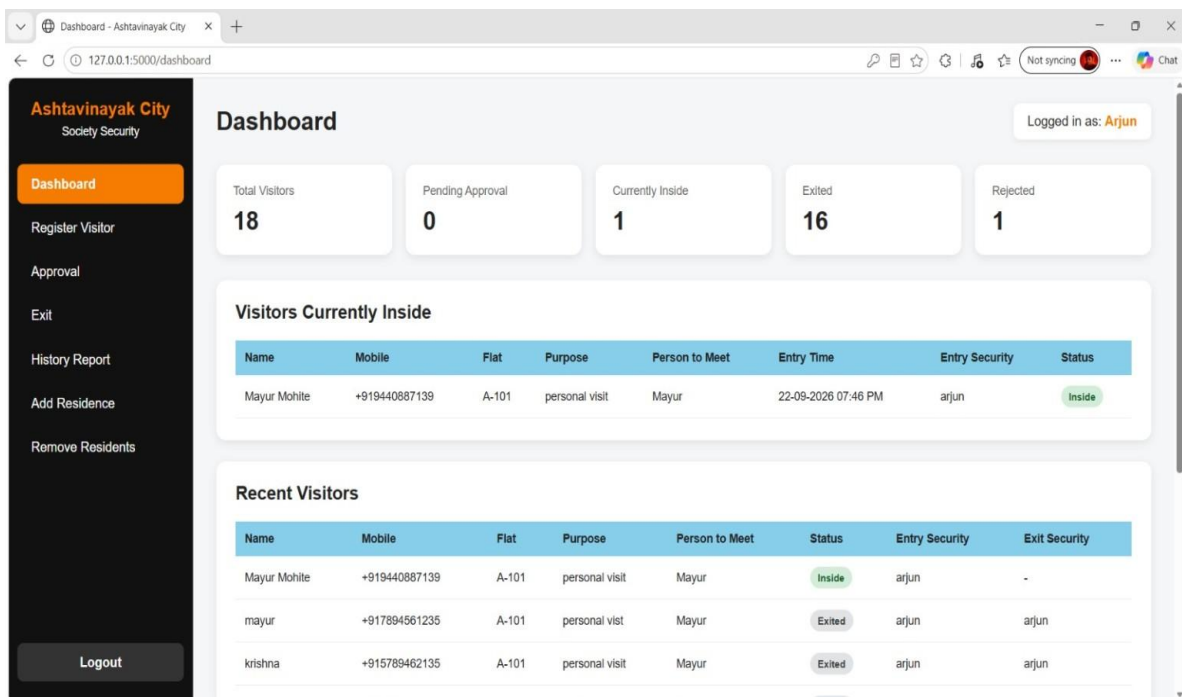
Username
Enter username

Password
Enter password

Login

Ashtavinayak City Security Management System

Figure 7.1: Security Login Page



Dashboard - Ashtavinayak City

Logged in as: Arjun

Dashboard

Register Visitor
Approval
Exit
History Report
Add Residence
Remove Residents
Logout

Summary Cards:

- Total Visitors: 18
- Pending Approval: 0
- Currently Inside: 1
- Exited: 16
- Rejected: 1

Visitors Currently Inside

Name	Mobile	Flat	Purpose	Person to Meet	Entry Time	Entry Security	Status
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	22-09-2026 07:46 PM	arjun	Inside

Recent Visitors

Name	Mobile	Flat	Purpose	Person to Meet	Status	Entry Security	Exit Security
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	Inside	arjun	-
mayur	+917894561235	A-101	personal visit	Mayur	Exited	arjun	arjun
krishna	+915789462135	A-101	personal visit	Mayur	Exited	arjun	arjun

Figure 7.2: Security Dashboard

Figure 7.3: Register Visitor Page

Figure 7.4: Visitor Phone Number Validation

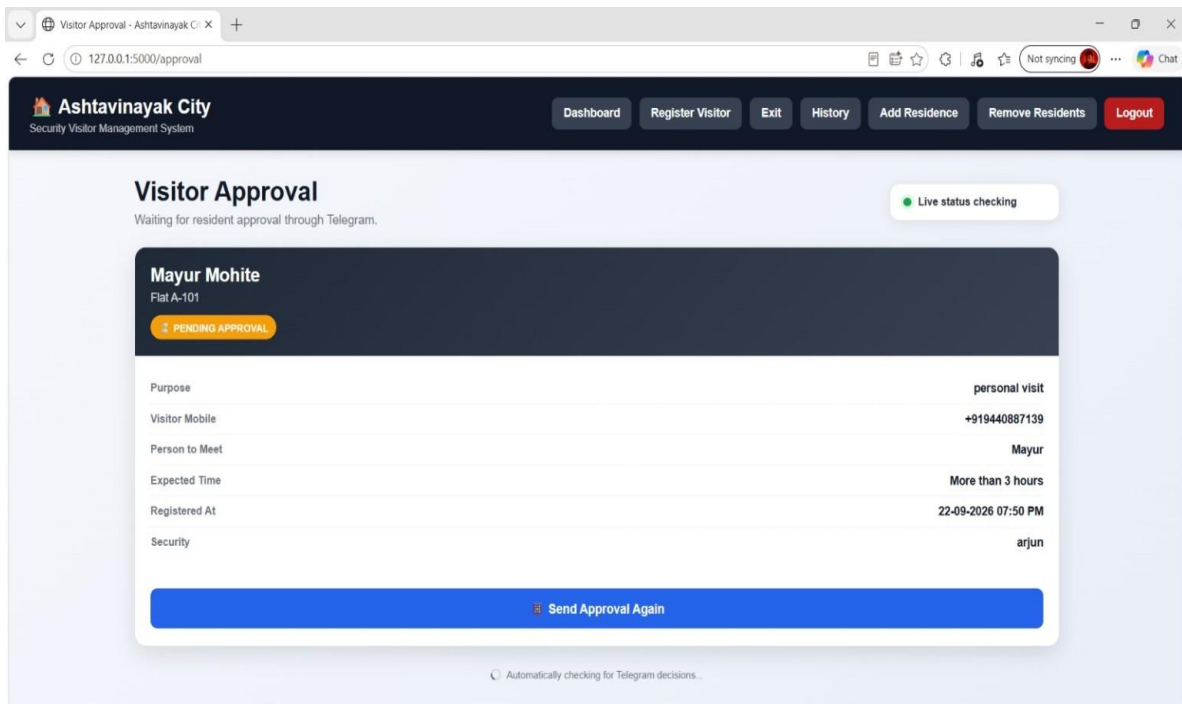


Figure 7.5: Visitor Approval Page

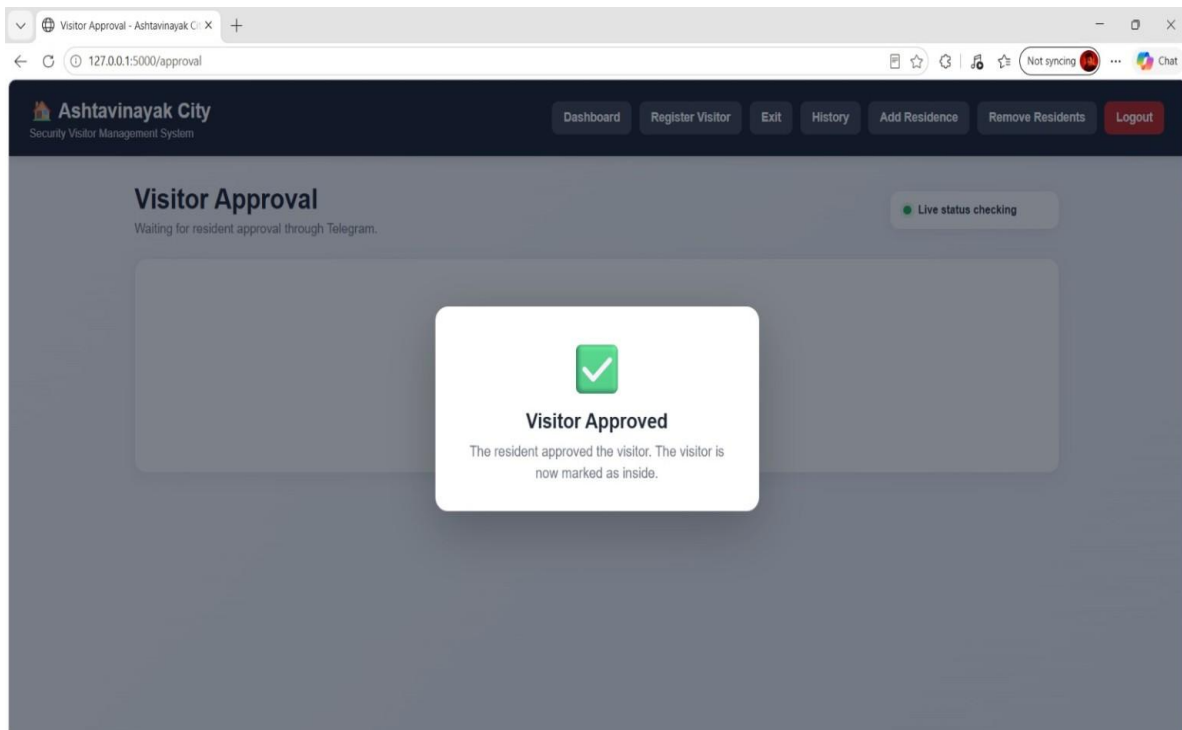


Figure 7.6: Visitor Approved Successfully



Figure 7.7: Telegram Visitor Approval Request

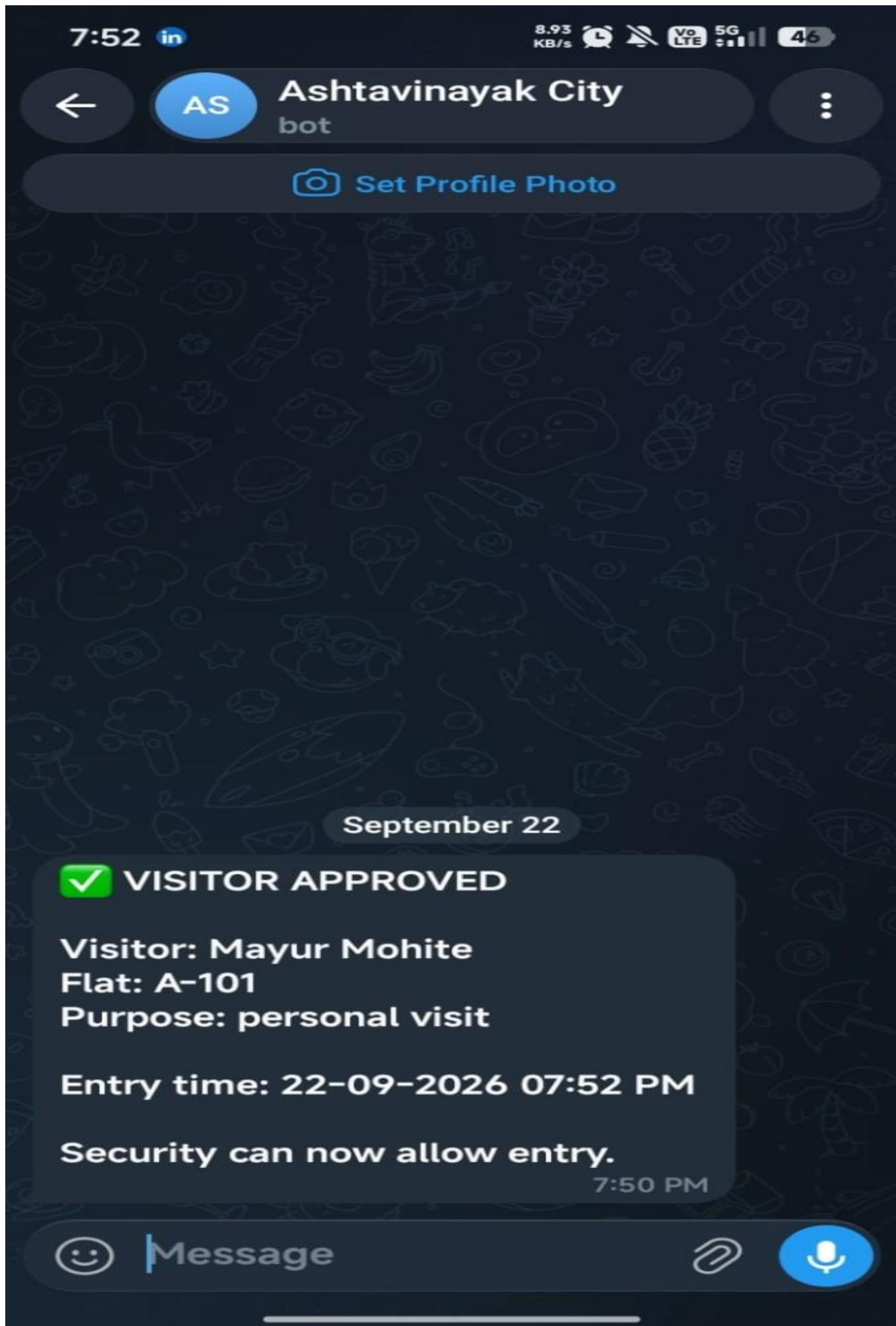


Figure 7.8: Telegram Visitor Approved Message

Exit - Ashtavinayak City

127.0.0.1:5000/exit

Not syncing

Chat

Ashtavinayak City

Society Security

Dashboard

Register Visitor

Approval

Exit

History Report

Add Residence

Remove Residents

Logout

Visitor Exit

Only the logged-in Security guard can record a visitor exit.

Visitors Currently Inside

Search Visitor

Search by flat, visitor name, mobile, purpose...

Example: A-101, C-111, visitor name, or mobile number

Visitor Name	Mobile	Flat	Purpose	Person to Meet	Entry Time	Entry Security	Status	Action
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	22-09-2026 07:52 PM	Arjun	Inside	Mark Exit
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	22-09-2026 07:52 PM	Arjun	Inside	Mark Exit
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	22-09-2026 07:46 PM	Arjun	Inside	Mark Exit

Exit Tracking — Security Only

When a visitor exits, the currently logged-in security guard is recorded as the Exit Security.

Example:

Visitor enters while Duryodhan is on duty.

Later, Duryodhan's duty ends and Sakuni starts duty.

If Sakuni records the exit, the history will show:

Figure 7.9: Visitor Exit Page

History Report - Ashtavinayak City

127.0.0.1:5000/history

Not syncing

Chat

Ashtavinayak City

Society Security

Dashboard

Register Visitor

Approval

Exit

History Report

Add Residence

Remove Residents

Logout

History Report

Complete visitor entry and exit history of Ashtavinayak City.

Visitor History

Name	Mobile	Flat	Purpose	Person to Meet	Status	Entry Time	Entry Security	Exit Time	Exit Security
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	Exited	22-09-2026 07:52 PM	Arjun	22-09-2026 07:53 PM	Arjun
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	Inside	22-09-2026 07:52 PM	Arjun	-	-
Mayur Mohite	+919440887139	A-101	personal visit	Mayur	Inside	22-09-2026 07:46 PM	Arjun	-	-
mayur	+917894561235	A-101	personal visit	Mayur	Exited	16-09-2026 02:09 PM	Arjun	16-09-2026 02:09 PM	Arjun
krishna	+915789462135	A-101	personal visit	Mayur	Exited	16-09-2026 01:20 PM	Arjun	16-09-2026 01:20 PM	Arjun
euwed	+917895462135	A-101	we3	Mayur	Exited	15-09-2026 06:18 PM	Arjun	16-09-2026 01:20 PM	Arjun
djhasdu	+917894561235	A-101	hgyuhjb	Mayur	Exited	15-09-2026 12:41 AM	Arjun	15-09-2026 12:42 AM	Arjun
dsd	+917894561235	A-101	wef	Mayur	Exited	11-09-2026 01:34 PM	Arjun	15-09-2026 12:42 AM	Arjun
kjadh	+917894561235	A-101	sede	Mayur	Exited	16-09-2026 01:20 PM	Arjun	16-09-2026 01:20 PM	Arjun
ertqerq	+911235647895	A-101	wcae	Mavur	Exited	10-09-2026 06:35 PM	Arjun	15-09-2026 12:42 AM	Arjun

Figure 7.10: Visitor History Report